

Tema 10

Archivos en C#



Índice

- Introducción
- Trabajando con archivos
- Archivos de texto
- Archivos binarios
- Serialización y deserialización
- Directorios
- Buenas prácticas para el manejo de archivos



- Hasta ahora, todos los programas que hemos desarrollado tienen un ciclo de vida que únicamente va desde que los iniciamos hasta que termina el método `Main`.
- Si queremos tener programas que puedan continuar en sucesivas ejecuciones, es necesario **persistir los datos** del programa.
- En la programación, los archivos son una forma esencial de **almacenar y recuperar datos** de forma persistente.

1. Introducción



- A diferencia de las variables y los datos en memoria que se pierden al cerrar una aplicación, los archivos permiten conservar información para su uso posterior, ofreciendo una solución para **almacenar datos a largo plazo**.
- Además de para persistir los datos de las variables únicamente, también necesitamos utilizar archivos para otro tipo de utilidades.

1. Introducción

¿Cuándo trabajar con archivos?

- Como hemos dicho, el uso de archivos es necesario en diversas aplicaciones, desde **guardar configuraciones de usuario y logs** de sistema, hasta almacenar bases de datos o documentos que necesitan ser consultados y modificados en distintos momentos.
- Vamos a ver algunas razones comunes para trabajar con archivos en C#.

1. Introducción

- **Persistencia de Datos:** Almacenar datos de manera que puedan ser recuperados incluso después de cerrar el programa. Es el caso más evidente donde vamos a utilizar los archivos.
- **Intercambio de Información:** Compartir información entre aplicaciones o sistemas a través de archivos. Por ejemplo, si presentamos datos en una gráfica, podemos leer esos datos desde un archivo de Excel.

1. Introducción

- **Control de estados:** Guardar el estado de una aplicación para permitir su recuperación más adelante. Por ejemplo, podemos almacenar las preferencias del usuario (desde el color del tema que prefiere a las credenciales de usuario y contraseña).
- **Generación de informes:** Crear archivos de texto o PDF que reporten los resultados o el estado de una aplicación.

1. Introducción

- Para trabajar de manera eficaz con archivos en C#, el espacio de nombres **System.IO** (tenemos que importarlo con un `using`) proporciona varias clases que permiten manejar diferentes aspectos de los archivos, desde su creación hasta su manipulación avanzada.
- Las clases principales que se usan para este propósito son **File** y **FileInfo**, ambas útiles para realizar tareas comunes como verificar si un archivo existe, crearlo, copiarlo, moverlo, o eliminarlo.

2. Trabajando con archivos

Clase File

- La clase `File` es estática y ofrece métodos directos para operar sobre archivos sin necesidad de instanciar un objeto.
- Esta clase es ideal para realizar operaciones rápidas y sencillas sobre archivos, pero carece de control avanzado sobre los atributos y detalles del archivo.
- Vamos a ver algunos de los métodos más comunes cuando trabajamos con la clase `File`.

2. Trabajando con archivos

- **File.Exists(string path):** Comprueba si un archivo existe en la ruta especificada.
- Retorna true si el archivo existe y false si no.
- Las rutas pueden ser absolutas o relativas. Recordad que si queremos poner el carácter '\' en un string hay que utilizar el carácter Unicode \\

2. Trabajando con archivos

```
1 if (!File.Exists("archivo.txt"))
2 {
3     Console.WriteLine("archivo.txt no se encuentra en la r
4 }
5
6 if(File.Exists("C:\\Windows\\notepad.exe"))
7 {
8     Console.WriteLine("Bloc de notas instalado");
```

- Las **rutas absolutas** son las que especifican la dirección completa de la carpeta en la que se encuentra un archivo, por ejemplo:
- Las **rutas relativas** tienen en cuenta la carpeta desde la que se está ejecutando el programa.
Para ver la ruta de nuestro programa, podemos hacer click derecho en el proyecto, dentro del *Explorador de soluciones* y elegir *Abrir carpeta en el explorador de archivos*. El programa se ejecuta por defecto en:

2. Trabajando con archivos

C:\Users\mruizgomez\Desktop\Curso C#\Tema 10\archivo.txt

bin\Debug\net 9.0\

- **File.Create(string path):** Crea un nuevo archivo en la ruta especificada.
- Si el archivo ya existe, se sobrescribirá.

2. Trabajando con archivos

```
1 File.Create("nuevoArchivo.txt");  
2  
3 File.Delete("nuevoArchivo.txt");
```

- **File.Delete(string path):** Elimina el archivo en la ruta especificada.
- Si el archivo no existe, no hace nada.

2. Trabajando con archivos

```
1 File.Delete("foto.jpg");  
2
```

- **File.Copy(string sourcePath, string destinationPath):** Copia un archivo de una ubicación a otra.
- Si el archivo de origen no existe, se lanzará una **excepción** de tipo `FileNotFoundException`.

2. Trabajando con archivos

```
1 try
2 {
3     File.Copy("data.zip", "C:\\users\\imagina\\Desktop\\backu
4 }
5 catch(FileNotFoundException ex)
6 {
7     Console.WriteLine("Archivo no encontrado. " + ex.Message)
```

- **File.Move(string sourcePath, string destinationPath)**: Mueve un archivo de una ubicación a otra, cambiando su ruta de acceso.
- Si el archivo de origen no existe, se lanzará una **excepción** de tipo `FileNotFoundException`.

2. Trabajando con archivos

```
1 try
2 {
3     File.Move("C:\\texto.txt", "C:\\Downloads\\texto.txt");
4 }
5 catch (FileNotFoundException ex)
6 {
7     Console.WriteLine("Archivo no encontrado. " + ex.Message)
```

Clase FileInfo

- FileInfo es una clase no estática que permite crear instancias para trabajar con un archivo específico.
- A diferencia de File, FileInfo proporciona métodos que operan en el contexto de un objeto de archivo específico, lo cual puede ser más eficiente en operaciones múltiples sobre el mismo archivo.

2. Trabajando con archivos

- Como no es una clase estática, debemos crear un objeto de esta clase para trabajar con él:
- Como parámetro debemos especificar la ruta del archivo, que puede ser relativa o absoluta.

2. Trabajando con archivos

```
1 FileInfo textFile = new FileInfo("archi
```

- Propiedades comunes de `FileInfo`:
- **Length**: Devuelve el tamaño del archivo en bytes.
- **CreationTime**: Indica la fecha y hora en que se creó el archivo.
- **LastAccessTime**: Devuelve la última fecha y hora en que se accedió al archivo.
- **Extension**: Obtiene la extensión del archivo, como `.txt`, `.jpg`, etc.

2. Trabajando con archivos

- Métodos de `FileInfo`:
- **`CopyTo(string destFileName)`**: Copia el archivo a la ruta especificada.
- **`MoveTo(string destFileName)`**: Mueve el archivo a una nueva ubicación.
- **`Delete()`**: Elimina el archivo especificado.

2. Trabajando con archivos

Liberación de recursos

- En C#, al trabajar con archivos, es importante **abrir y cerrar** los archivos correctamente.
- En el caso de la clase `FileInfo`, si abrimos un archivo utilizando `Open`, debemos asegurarnos de que lo cerramos mediante `Close`.
- Cuando tengamos bloques `try-catch`, debemos asegurarnos de liberar los ficheros abiertos añadiendo un `finally`.

2. Trabajando con archivos

- Para el resto de clases de manejo de archivo que vamos a ver, como `StreamReader`, `BinaryWriter` o `FileStream`, se recomienda utilizar un bloque **using**, que define una variable únicamente en su ámbito y la libera correctamente cuando finaliza:

2. Trabajando con archivos

```
1 using(StreamReader txtReader = new StreamReader("a
2 {
3     // Operaciones con el archivo
4 }
5
6 // Se libera automáticamente al acabar el bloque
```

Errores comunes al trabajar con archivos

- Cuando trabajamos con archivos, hay una serie de excepciones que nos pueden ocurrir.
- Son especialmente comunes 3 de ellas:
 - `FileNotFoundException`
 - `IOException`
 - `UnauthorizedAccessException`

2. Trabajando con archivos

- **FileNotFoundException:** Ocurre cuando intentamos abrir o trabajar con un archivo que no existe. Es decir, no se encuentra en la dirección que hemos especificado como ruta.
- **UnauthorizedAccessException:** Ocurre cuando no se tienen permisos para acceder o modificar el archivo con el que queremos trabajar.
- **IOException:** Error de E/S. Ocurre habitualmente cuando el archivo está en uso por otro proceso (o no se ha liberado correctamente).

2. Trabajando con archivos

Vídeo práctico

2. Trabajando con archivos

Vídeo práctico - código

2. Trabajando con archivos

```
1  static void Main(string[] args)
2  {
3      var relativeFile = File.Exists("Carpeta\\ejemplo.
4      var absoluteFile = File.Exists("C:\\Curso C#\\rut
5
6      Console.WriteLine(relativeFile);
7      Console.WriteLine(absoluteFile);
8
9      //File.Create("nuevoArchivo.jpg");
10     File.Create("C:\\Curso C#\\nuevo.txt");
11     File.Delete("nuevoArchivo.jpg");
12     //File.Copy("C:\\Curso C#\\ruta.txt", "ruta.txt")
13     //File.Move("ruta.txt", "Carpeta\\ruta.txt");
14
15     File.Create("C:\\prueba.txt");
16
17     FileInfo txtFile = new FileInfo("Carpeta\\ruta.tx
18     Console.WriteLine(txtFile.FullName);
19     Console.WriteLine(txtFile.Length);
20     Console.WriteLine(txtFile.CreationTime);
21
22     . . . . .
```

- En C#, el manejo de archivos de texto es una tarea común que permite a los desarrolladores leer y escribir información de manera sencilla y eficiente.
- Los archivos de texto son útiles para almacenar datos en un formato legible para humanos, lo que resulta ideal para tareas como guardar configuraciones, logs o cualquier tipo de información estructurada de manera simple.
- C# ofrece varias clases dentro del espacio de nombres `System.IO` para la manipulación de archivos de texto

3. Archivos de texto

- Para trabajar con archivos de texto, podemos utilizar las clases:
- **File**: Aparte de los métodos comunes que hemos visto, tenemos métodos específicos para trabajar con archivos de texto.
- **StreamReader**: Clase específica para leer archivos de texto con mayor control sobre el proceso.
- **StreamWriter**: Clase específica para escribir archivos de texto que ofrece más control y opciones.

3. Archivos de texto

Lectura de Archivos de Texto

- Como acabamos de ver, tenemos diferentes opciones para leer archivos de texto.
- La clase **File** nos ofrece 2 métodos específicos para la lectura de archivos de texto.
- También podemos usar la clase **StreamReader** para tener más control sobre la lectura.

3. Archivos de texto

- Vamos a suponer para los ejemplos el siguiente archivo de texto, llamado `dumpstack.txt` y ubicado en la misma carpeta que nuestro programa:

3. Archivos de texto

```
DLOGFILE00010000DUMP
Dump stack initialized at UTC: 2024/08/12 15:57:05, local time: 20
#BugCheckCode 0x00000000000000009F
#BugCheckP1 0x000000000000000003
#BugCheckP2 0xFFFFCF01974A4360
#BugCheckP3 0xFFFFFA8F96CD7738
#BugCheckP4 0xFFFFCF019D83EBE0
Progress 0x00000042
Elapsed BugCheck duration 00001303ms
Starting get secondary dump callbacks size.
Progress 0x00000052
Finish get secondary dump callbacks size.
Dump Type: 4, Total Dump Size: 8922246, Secondary Dump Size: 87025
Starting write of dump header.
Finish write of dump header.
Starting write of minidump data.
```

- **`File.ReadAllText(string path)`**: Lee todo el contenido del archivo de una sola vez y lo devuelve como una cadena.

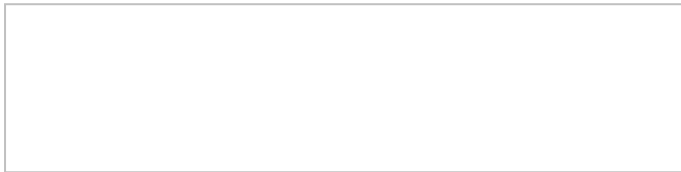
3. Archivos de texto

```
1 string fileText = File.ReadAllText("dumpstack  
2 Console.WriteLine(fileText);
```

- **File.ReadAllLines(string path):** Lee cada línea del archivo y devuelve un array de string, donde cada elemento representa una línea del archivo.

3. Archivos de texto

```
1 string[] fileLines = File.ReadAllLines("dumpstac  
2  
3 Console.WriteLine(fileLines[5]);
```



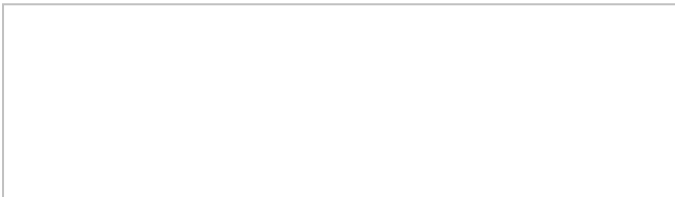
- **StreamReader**: Clase que permite crear un objeto para leer un archivo de texto.
- Inicializamos el objeto con `new StreamReader(string ruta)` y después tenemos una serie de operaciones a nuestra disposición.
- Vamos a ver las principales operaciones.

3. Archivos de texto

- **Read()**: Devuelve un `int` con el código Unicode del siguiente carácter del archivo.
- Además, avanza la posición actual de lectura.
- Cuando hemos leído todo el archivo, devuelve -1.

3. Archivos de texto

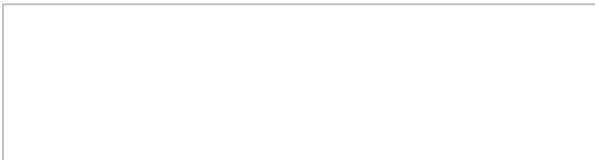
```
1 using(StreamReader txtReader = new StreamReader("di
2 {
3     Console.WriteLine(txtReader.Read());
4     Console.WriteLine((char)txtReader.Read());
5 }
```



- **Read(char[] buffer, int index, int count):** Rellena en el array de char pasado como parámetro los siguientes 'count' caracteres a partir de 'index'.

3. Archivos de texto

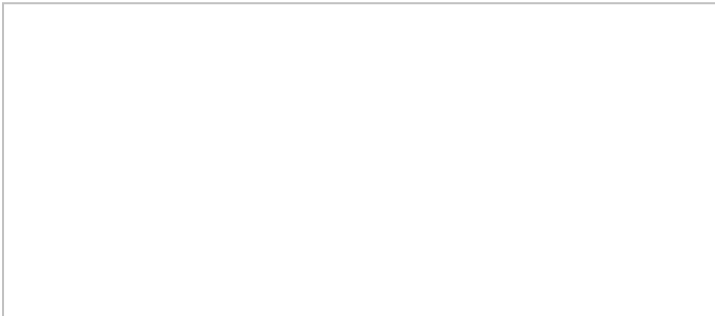
```
1  using(StreamReader txtReader = new StreamReader("dl
2  {
3      char[] charArray = new char[10];
4
5      txtReader.Read(charArray, 0, 10);
6
7      foreach (char c in charArray)
8      {
9          Console.Write(c);
10     }
```



- **Peek()**: Devuelve un `int` con el siguiente caracter a leer, pero **no avanza** la posición.

3. Archivos de texto

```
1 using(StreamReader txtReader = new StreamReader("dl
2 {
3     Console.WriteLine((char)txtReader.Peek());
4     Console.WriteLine((char)txtReader.Peek());
5     Console.WriteLine((char)txtReader.Peek());
6     Console.WriteLine((char)txtReader.Read());
7     Console.WriteLine((char)txtReader.Read());
8 }
```



- **ReadLine()**: Devuelve un `string` con el texto comprendido desde la posición actual hasta el siguiente salto de línea.
- Cuando hemos leído todo el archivo, devuelve `null`.

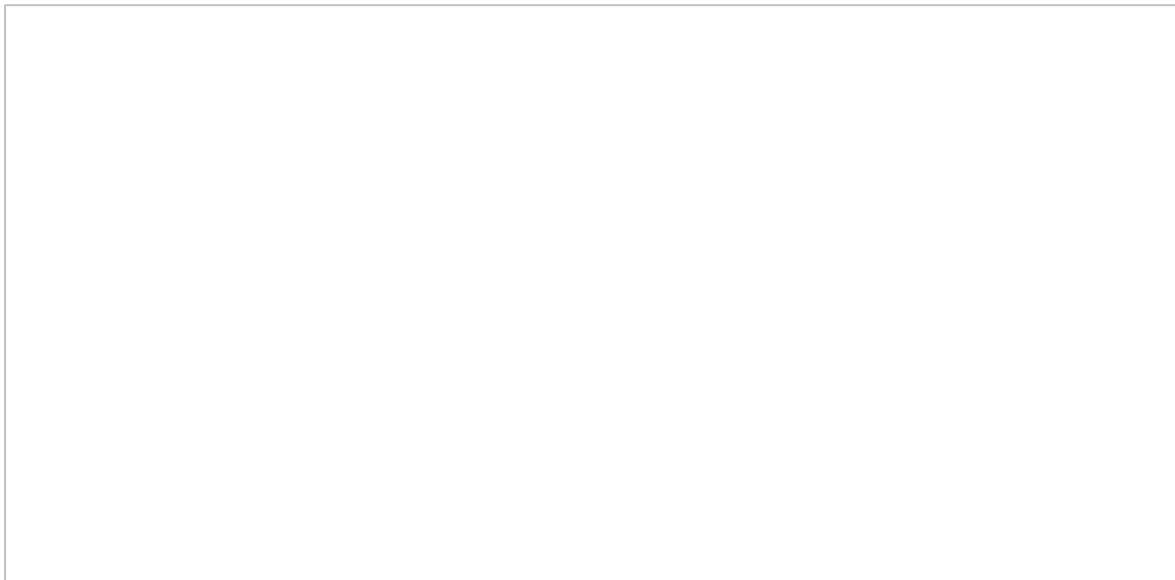
3. Archivos de texto

```
1 using(StreamReader txtReader = new StreamReader("dui
2 {
3     Console.WriteLine(txtReader.ReadLine());
4     Console.WriteLine(txtReader.ReadLine());
5     Console.WriteLine(txtReader.ReadLine());
```

- **ReadToEnd()**: Devuelve un string con el texto comprendido desde la posición actual hasta el final del archivo.

3. Archivos de texto

```
1 using(StreamReader txtReader = new StreamReader("du
2 {
3     Console.WriteLine(txtReader.ReadToEnd());
```



Escritura de archivos de texto

- Para escribir texto en un archivo, utilizaremos la clase **StreamWriter**.
- Con **StreamWriter**, podemos escribir datos en un archivo de texto línea a línea o carácter a carácter, con facilidad.

3. Archivos de texto

- Para abrir y escribir en un archivo, debemos crear una **instancia** de `StreamWriter`.
- Al crear esta instancia, es posible especificar si se desea sobrescribir el archivo existente o añadir contenido al final del archivo.
- El constructor de `StreamWriter` acepta un parámetro `append` que, cuando es `true`, añade el nuevo contenido al final del archivo sin sobrescribir el contenido existente y cuando es `false` o se omite, el archivo se sobrescribe con el nuevo contenido.

3. Archivos de texto

Ejemplo

- En este caso, el archivo *resultados.txt* será sobrescrito, perdiendo todo su contenido.
- En el segundo caso, el texto que añadamos se escribirá al final del que ya contenía el archivo.

3. Archivos de texto

```
1 using(StreamWriter streamWriter = new StreamWriter("resl  
2 {  
3     // Operaciones de escritura  
4 }
```

```
1 using(StreamWriter streamWriter = new StreamWriter("resulta
```

```
2 {  
3
```

```
// Operaciones de escritura
```

- Una vez que tenemos nuestra **instancia** de tipo `StreamWriter`, podemos trabajar con ella.
- La clase `StreamWriter` incluye varios métodos para escribir en el archivo de manera flexible.
- Vamos a ver los más importantes.

3. Archivos de texto

- **Write():** Escribe un dato al archivo sin agregar un salto de línea. Puede aceptar datos de diferentes tipos como string, char, int...

3. Archivos de texto

```
1 using (StreamWriter writer = new StreamWriter("miA
2 {
3     writer.Write("Texto en la misma línea. ");
4     writer.Write(123);
```

- **WriteLine():** Escribe el dato especificado seguido de un salto de línea.
- Esto es útil para escribir líneas completas de texto en el archivo, lo que facilita la lectura posterior.

3. Archivos de texto

```
1 using (StreamWriter writer = new StreamWriter("miA
2 {
3     writer.WriteLine("Primera línea.");
4     writer.WriteLine('@');
```

- **Flush()**: Fuerza la escritura de todos los datos en el buffer del StreamWriter al archivo.
- Es útil cuando se quiere asegurar que el contenido se escribe inmediatamente en el disco, sin esperar a que se cierre el flujo.

3. Archivos de texto

```
1 using (StreamWriter writer = new StreamWriter("miArchivo.txt"))
2 {
3     writer.WriteLine("Texto guardado inmediatamente.");
4     writer.Flush(); // Se asegura de que los datos se escriban en el a
5 }
```

Vídeo práctico

3. Archivos de texto

Vídeo práctico - código

3. Archivos de texto

```
1  static void Main(string[] args)
2  {
3      using (StreamReader reader = new StreamReader("ej
4      {
5          for (int i = 0; i < 10; i++)
6          {
7              int charNumber = reader.Read();
8              char nextChar = (char)charNumber;
9
10             Console.WriteLine("{0} ({1})", nextChar,
11             }
12
13             char[] charArray = new char[20];
14
15             reader.Read(charArray, 0, charArray.Length);
16             Console.WriteLine(charArray);
17
18             string nextLine = reader.ReadLine();
19             Console.WriteLine(nextLine);
20
21             nextLine = reader.ReadLine();
22             Console.WriteLine(nextLine);
```

¿Qué es un archivo binario?

- Un archivo binario es un archivo que contiene datos en **formato binario**, lo que significa que la información se almacena en forma de bits y no en caracteres legibles para humanos.
- Este tipo de archivo es ideal para almacenar datos estructurados, como números, objetos, o conjuntos de datos complejos, de forma más eficiente y sin ocupar tanto espacio como un archivo de texto.

4. Archivos binarios

- Al contrario que los archivos de texto, este tipo de archivos **no son legibles** para el usuario.
- En C#, el espacio de nombres `System.IO` proporciona las clases `BinaryWriter` y `BinaryReader` para escribir y leer datos binarios de manera sencilla.

4. Archivos binarios

Ventajas de los Archivos Binarios

- **Eficiencia de Espacio:** Los archivos binarios suelen ser más pequeños que los archivos de texto, ya que no contienen caracteres adicionales ni requieren formato de texto.
- **Velocidad de Procesamiento:** El procesamiento de archivos binarios suele ser más rápido porque el sistema no necesita convertir datos en texto.
- **Ideal para Datos Complejos:** Los archivos binarios pueden almacenar datos de tipos complejos, como enteros, flotantes, estructuras de datos u objetos.

4. Archivos binarios

Escritura de archivos binarios

- La clase `BinaryWriter` permite escribir datos en formato binario. Su uso es similar al de `StreamWriter`, pero opera con datos en bruto.
- Para crear una instancia de `BinaryWriter`, debemos combinar el constructor con la función `File.Open`.
- Una vez abierto, utilizamos el método `Write` para escribir cualquier tipo válido que necesitemos.

4. Archivos binarios

Ejemplo

- Utilizamos `File.Open` para abrir el fichero `datos.bin`. En este caso, el fichero lo vamos a crear, no es necesario que exista.
- Posteriormente, escribimos datos en el archivo que hemos abierto.

4. Archivos binarios

```
1 using (BinaryWriter writer = new BinaryWriter(File.Open("datos.bin",  
2 {  
3     writer.Write(42);  
4     writer.Write(double.Pi);  
5     writer.Write("Hola, mundo");
```

- La enumeración `FileMode` nos permite especificar el modo de apertura del fichero.
- Tenemos disponibles varias opciones como crear un archivo nuevo, abrir uno existente y sobrescribir, abrir uno existente y añadir los datos al final....
- Podemos consultar todas las opciones en la documentación oficial de Microsoft:

4. Archivos binarios

<https://learn.microsoft.com/es-es/dotnet/api/system.io.filemode?view=net-9.0>

- Mediante el uso de `Write`, podemos guardar valores de cualquier tipo numérico primitivo, `bool`, `string`, `char`...
- Esto nos permite guardar el estado de un objeto para poder leerlo después.

4. Archivos binarios

- Por ejemplo, si suponemos la siguiente clase Car:

4. Archivos binarios

```
1  public class Car
2  {
3      ...
4
5      public string Brand { get; set;
6      public string Model { get; set;
7      public short Year { get; set; }
8      public int Kilometers { get; set
9
10     ...
11 }
```

- Podemos guardar en un fichero el estado de una instancia de dicha clase:

4. Archivos binarios

```
1  Car clio = new Car()  
2  {  
3      Brand = "Renault",  
4      Model = "Clio",  
5      Year = 2015,  
6      Kilometers = 185408  
7  };  
8  
9  using (BinaryWriter writer = new BinaryWriter(File.Open("clio.bin",  
10 {  
11     writer.Write(clio.Brand);  
12     writer.Write(clio.Model);  
13     writer.Write(clio.Year);  
14     writer.Write(clio.Kilometers);
```

4. Archivos binarios

Lectura de archivos binarios

- Para leer datos binarios utilizamos la clase `BinaryReader`.
- Esta permite recuperar los datos en el mismo orden en que fueron escritos.
- También debemos combinarla con `File.Open` para crear una instancia de esta clase.

4. Archivos binarios

Ejemplo

- En este caso, utilizaremos `FileMode.Open` como modo habitual, ya que si utilizamos otros como `Create` podemos borrar los datos que contiene el fichero.

```
1 using (BinaryReader reader = new BinaryReader(File.Open("clio.bin")
2 {
3     // Operaciones de lectura
4 }
```

4. Archivos binarios

- Una vez que tenemos abierto nuestro fichero, podemos leer los datos que contiene.
- Para leer correctamente la información, debemos saber qué tipo de dato nos toca leer.
- Tenemos disponible una gran cantidad de métodos para leer cada tipo de dato específico.

4. Archivos binarios

- Algunos de los más comunes son los siguientes:
- Cada lectura hace avanzar la posición el número de bytes que ocupa el tipo leído.
- Tenemos muchos otros métodos que podemos ver con el IntelliSense de Visual Studio.

Método BinaryReader	Descripción
ReadInt32()	Lee un entero de 4 bytes (int)

ReadDouble()	Lee un valor de tipo double
ReadBoolean()	Lee un valor bool
ReadString()	Lee una cadena en formato binario
ReadByte()	Lee un valor de tipo byte

4. Archivos binarios

- Teniendo en cuenta la clase Car y el archivo que guardamos en el punto anterior, vamos a reconstruir el objeto:

```
1 Car newCar = new Car();  
2  
3 using (BinaryReader reader = new BinaryReader(File.Open("clio.bin"  
4 {  
5     newCar.Brand = reader.ReadString();  
6     newCar.Model = reader.ReadString();  
7     newCar.Year = reader.ReadInt16();  
8     newCar.Kilometers = reader.Read();  
9 }
```

4. Archivos binarios

- Una vez leído el archivo, el objeto newCar tiene las mismas propiedades que el objeto que guardamos en dicho archivo.
- Como vemos en el ejemplo, es totalmente necesario saber en qué orden guardamos las propiedades y de qué tipo de dato es cada una.

4. Archivos binarios

Vídeo práctico

4. Archivos binarios

Vídeo práctico - código

```
1 namespace ConsoleApp1
2 {
3     public class Product
4     {
5         public string Name { get; set; }
6         public double Price { get; set; }
7         public int Stock { get; set; }
8         public bool IsAvailable { get; set; }
9     }
10
11     public class Program
12     {
13         static void Main(string[] args)
14         {
15             //List<Product> products = new List<Product>
16             //{
17                 // new Product() { Name = "Impresora", P
18                 // new Product() { Name = "Teclado", Pri
19                 // new Product() { Name = "Portátil", Pr
20                 // new Product() { Name = "Monitor", Pri
21             //};
22         }
```

5. Serialización y deserialización

- Como acabamos de ver, el guardado y recuperación de objetos complejos mediante el uso de archivos binarios es bastante complejo.
- Debemos tener en cuenta el orden y el tipo de datos y un error hará que el objeto reconstruido no se corresponda con el original.
- Además, es poco flexible ante cambios en la estructura de las clases que guardamos.

5. Serialización y deserialización

- La serialización y deserialización son técnicas que permiten convertir objetos en datos que pueden ser almacenados o transmitidos y luego reconstruidos en su forma original de manera mucho más sencilla.
- La serialización es, pues, el proceso de convertir un objeto en un flujo de bytes o una cadena de texto (como JSON o XML) para **almacenar el estado del objeto** en un archivo, base de datos o transmitirlo a través de una red.

5. Serialización y deserialización

- Esto es útil para:
 - Guardar datos en archivos para un acceso posterior.
 - Enviar datos a través de redes.
 - Clonar objetos.
- En C#, el espacio de nombres `System.Runtime.Serialization` proporciona soporte para la serialización binaria (que no vamos a ver), mientras que `System.Text.Json` y `System.Xml.Serialization` son específicos para los formatos JSON y XML de serialización en texto.

5. Serialización y deserialización

Formato JSON

- El formato JSON es ampliamente utilizado para la comunicación de datos entre aplicaciones web debido a su formato legible y ligero.
- En C#, vamos a trabajar con la clase estática **JsonSerializer** del espacio de nombres `System.Text.Json` para realizar la serialización y deserialización de objetos en JSON.

5. Serialización y deserialización

- Para **serializar** un objeto en JSON, utilizamos el método `JsonSerializer.Serialize(objeto)`.
- Este método nos devuelve un `string` con el resultado de la serialización en formato JSON.
- Podemos guardar este `string` a un archivo de texto para persistir nuestros datos cuando cerremos el programa.

5. Serialización y deserialización

Ejemplo

- En este ejemplo, la cadena `clioJson` almacena el valor:

```
1 Car clio = new Car()  
2 {  
3     Brand = "Renault",  
4     Model = "Clio",  
5     Year = 2015,  
6     Kilometers = 185408  
7 };  
8  
9 string clioJson = JsonSerializer.Serialize
```

```
{"Brand":"Renault","Model":"Clio","Year":2015,"Kilometers":185408}
```

5. Serialización y deserialización

- Para **deserializar** un objeto en JSON, utilizamos el método `JsonSerializer.Deserialize<Tipo>(cadena)`.
- Este método nos devuelve un objeto con el resultado de la deserialización de la cadena JSON pasada como parámetro.

5. Serialización y deserialización

Ejemplo

- En este ejemplo, pasamos la cadena que obtuvimos al serializar y la convertimos a un objeto de tipo Car.
- Este objeto tiene las mismas propiedades que el objeto serializado.

```
1 string jsonString = {"Brand":"Renault","Model":"Clio","Year":2015,"Ki"  
2  
3 Car deserializedClio = JsonSerializer.Deserialize<Car>(jsonString);
```

5. Serialización y deserialización

- Mediante la serialización JSON también podemos almacenar colecciones y objetos compuestos.
- Por ejemplo, si serializamos el objeto `Shelter` del ejercicio 1 del tema 6, obtendremos una cadena como esta:

```
{"Pets":[{"Name":"Ubay","Species":"Perro"},  
{"Name":"Michi","Species":"Gato"}],"People":  
[{"Name":"Juan","Id":55,"Pets":[]},  
{"Name":"Antonio","Id":132,"Pets":  
[{"Name":"Rufo","Species":"Perro"}]},  
{"Name":"Mar\u00EDA","Id":27,"Pets":[]}]}
```

5. Serialización y deserialización

- Por defecto, la serialización de un objeto incluirá todas las propiedades públicas de la clase.
- Podemos elegir no incluir alguna de ellas o también incluir campos privados a la hora de serializar.
- También podemos cambiar el nombre con el que se guarda en la cadena de texto una propiedad.
- Para ello utilizamos los modificadores `[JsonIgnore]`, `[JsonInclude]` o `[JsonPropertyName("Nombre")]`, respectivamente.

5. Serialización y deserialización

Ejemplo

- Añadimos estos modificadores a la clase Car, incluyendo un campo privado nuevo e ignorando el campo Year. El resultado ahora será:

```
1 public class Car
2 {
3     [JsonInclude]
4     private int _horsePower = 115;
5     public string Brand { get; set; }
6     public string Model { get; set; }
7     [JsonIgnore]
8     public short Year { get; set; }
9     [JsonPropertyName("Km")]
```

```
10 public int Kilometers { get; set; }
```

```
{"Brand":"Renault","Model":"Clio","Km":185408,"_horsePower":115}
```

5. Serialización y deserialización

Vídeo práctico

5. Serialización y deserialización

Vídeo práctico - código

```
1  public class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5      public int Stock { get; set; }
6      public bool IsAvailable { get; set; }
7  }
8
9  public class Program
10 {
11     static void Main(string[] args)
12     {
13         List<Product> products = new List<Product>()
14         {
15             new Product() { Name = "Impresora", Price =
16             new Product() { Name = "Teclado", Price = 4
17             new Product() { Name = "Portátil", Price =
18             new Product() { Name = "Monitor", Price = 1
19         };
20
21         string jsonProduct = JsonSerializer.Serialize(p
22
```

5. Serialización y deserialización

Formato XML

- Además de JSON, el otro formato más común para serializar datos como texto es el formato XML.
- Aunque a nivel de uso es muy similar, presenta algunas diferencias respecto a JSON a nivel estructural.
- Como hemos visto, JSON almacena los datos como pares "clave-valor", sin comentarios o indentación.

5. Serialización y deserialización

- XML por contra, almacena los datos en una estructura jerárquica, estructurada, con soporte para comentarios y metadatos y que admite validación de los datos.
- El formato XML es muy utilizado en aplicaciones empresariales y también en sistemas antiguos, mientras que JSON domina en el campo de las aplicaciones web y transmisión de datos en APIs.

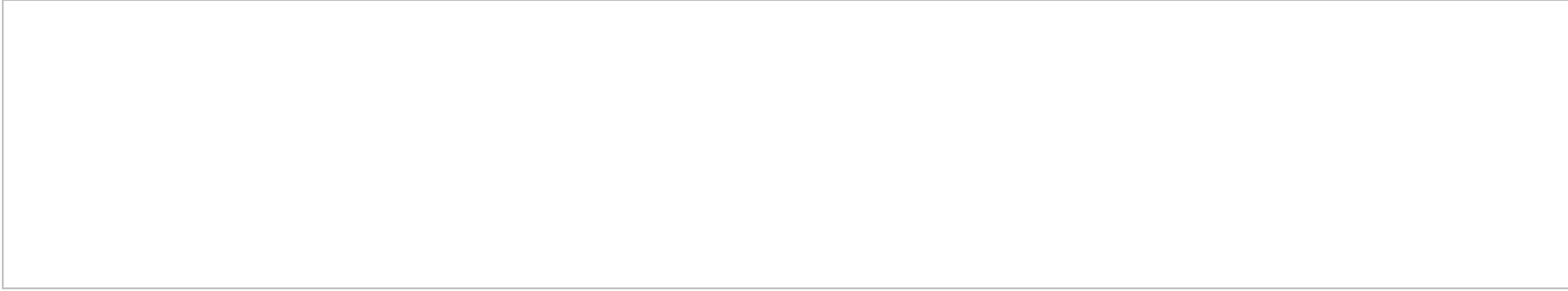
5. Serialización y deserialización

- Para serializar y deserializar en formato XML, C# utiliza la clase `XmlSerializer` del espacio de nombres `System.Xml.Serialization`.
- Al contrario que `JsonSerializer`, la clase `XmlSerializer` no es estática, por lo que tendremos que instanciarla para trabajar con ella.
- Además, necesitamos un objeto `StringWriter` al que escribir la información, no nos devuelve la cadena directamente.

5. Serialización y deserialización

Ejemplo:

```
1 XmlSerializer serializer = new XmlSerializer(typeof(Tipo));
2 using (StringWriter writer = new StringWriter())
3 {
4     serializer.Serialize(writer, clio);
5     string xmlString = writer.ToString();
6     Console.WriteLine(xmlString);
7 }
```



5. Serialización y deserialización

- Para deserializar una cadena XML, utilizamos también un objeto `XmlSerializer`.
- En este caso, debemos pasarle un `StringReader` para realizar la operación:

```
1 using (StringReader reader = new StringReader(xmlString))
2 {
3     Car xmlCar = (Car)serializer.Deserialize(reader);
4     Console.WriteLine("Deserializado coche: {0} {1}", xmlCar.Brand, xmlCar.Model);
5 }
```

5. Serialización y deserialización

- Como hemos visto, es algo más complicado trabajar con XML que con JSON.
- Además, el formato JSON es la tendencia actual, mientras el XML queda reservado para sistemas que necesitan compatibilidad con este tipo de archivos.
- Así pues, utilizaremos la serialización JSON como formato por defecto cuando lo necesitemos en nuestros proyectos.

5. Serialización y deserialización

Vídeo práctico

5. Serialización y deserialización

Vídeo práctico - código

```

1  namespace ConsoleApp1
2  {
3      public class Product
4      {
5          public string Name { get; set; }
6          public double Price { get; set; }
7          public int Stock { get; set; }
8          public bool IsAvailable { get; set; }
9      }
10
11     public class Program
12     {
13         static void Main(string[] args)
14         {
15             List<Product> products = new List<Product>()
16             {
17                 new Product() { Name = "Impresora", Price = 450, Stock = 10, IsAvailable = true },
18                 new Product() { Name = "Teclado", Price = 45, Stock = 10, IsAvailable = true },
19                 new Product() { Name = "Portátil", Price = 1200, Stock = 10, IsAvailable = true },
20                 new Product() { Name = "Monitor", Price = 120, Stock = 10, IsAvailable = true },
21             };
22
23             YmlSerializer serializer = new YmlSerializer(tv

```

5. Serialización y deserialización

Deserialización de interfaces

- Cuando queremos serializar y deserializar interfaces nos encontramos con un problema.
- Esto es debido a que el sistema no sabe qué clase debe instanciar y debemos proporcionar esa información nosotros.
- En el formato XML es algo complejo mientras que en JSON es más sencillo.

5. Serialización y deserialización

- Serializar una interfaz en JSON es directo, no hay que hacer nada ya que no guarda información de clases en la cadena, al contrario que el XML.
- Sin embargo, sí que debemos especificar la clase a la que vamos a deserializar dicha interfaz.
- Para ello, debemos crear un constructor en el que especifiquemos el tipo al que queremos asignar la propiedad de la interfaz.

5. Serialización y deserialización

Ejemplo:

```
1  public interface IEngine
2  {
3      int HP { get; set; }
4      int CC { get; set; }
5      int Kilometers { get; set; }
6  }
7
8  public class CarEngine : IEngine
9  {
10     public int HP { get; set; }
11     public int CC { get; set; }
12     public int Kilometers { get; set; }
13 }
14
15 public Car() { }
16
17 public Car(CarEngine engine)
18 {
19     Engine = engine;
20 }
21
22 ...
23
24 public IEngine Engine { get; set; }
```

5. Serialización y deserialización

- En este ejemplo, nos creamos una interfaz `IEngine` y una clase que la implementa `CarEngine` (podemos tener otras distintas) y añadimos una propiedad de dicha interfaz a la clase `Car`.
- A la hora de deserializar, debemos añadir el constructor que especifique qué clase concreta va a instanciar la interfaz (en este caso, `CarEngine`).
- Para ello, el constructor recibe un parámetro de la clase y lo asigna a la propiedad. De esta sencilla manera, el sistema ya sabe deserializar el JSON.

6. Directorios

- Además de archivos, C# también permite gestionar y manipular las carpetas del sistema de archivos.
- Las clases **Directory** y **DirectoryInfo** proporcionan los métodos y propiedades necesarios para crear, eliminar, mover, y consultar directorios.
- Estas clases se encuentran en el espacio de nombres `System.IO`, y su uso es esencial para aplicaciones que necesitan leer y organizar archivos de manera dinámica.

6. Directorios

Clase Directory

- La clase estática `Directory` ofrece métodos para manipular directorios sin la necesidad de instanciar un objeto, lo que es conveniente para operaciones directas y rápidas.
- Vamos a ver la lista de los métodos más comunes de esta clase.

6. Directorios

- **CreateDirectory(string path):** Crea un nuevo directorio en la ruta especificada. Si la ruta incluye subdirectorios inexistentes, los crea automáticamente.
- **Delete(string path, bool recursive):** Elimina el directorio especificado en la ruta path. Si recursive es true, elimina también todos los subdirectorios y archivos en el directorio.
- **Exists(string path):** Verifica si el directorio especificado existe, devolviendo true o false.

6. Directorios

- **GetFiles(string path):** Devuelve una lista de los archivos contenidos en el directorio especificado en formato array de `string`, donde cada cadena contiene la ruta del archivo. También podemos especificar un filtro y un modo de búsqueda (buscar en todas las subcarpetas, sólo en la raíz...).
- **GetDirectories(string path):** Devuelve una lista de subdirectorios dentro del directorio indicado en formato array de `string`. También podemos especificar un filtro y un modo de búsqueda (recursivo, sólo 1 nivel...).

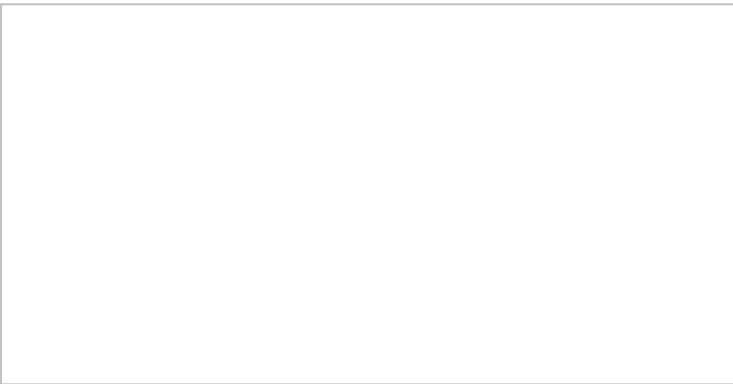
6. Directorios

Ejemplo

```
1  string[] files = Directory.GetFiles("C:\\tts55", "*.*",
2
3  foreach (string file in files)
4  {
5      Console.WriteLine(file);
6  }
7
8  Console.WriteLine();
9
10 Directory.CreateDirectory("C:\\tts55\\Nueva Carpeta");
11
12 string[] directories = Directory.GetDirectories("C:\\tt
13
14 foreach (string directory in directories)
15 {
16     Console.WriteLine(directory);
17
18     string[] innerDirectories = Directory.GetDirectorie
19
20     foreach (string innerDirectory in innerDirectories)
21     {
```

6. Directorios

- En este ejemplo, listamos todos los archivos de la carpeta especificada.
- Posteriormente creamos una carpeta y listamos de manera recursiva las carpetas que existen en la ruta, añadiendo tabulaciones para ver las subcarpetas.
- Posteriormente eliminamos el directorio completo.



6. Directorios

Clase DirectoryInfo

- A diferencia de `Directory`, `DirectoryInfo` es una clase que permite crear instancias y, por lo tanto, trabajar con un objeto `DirectoryInfo` específico.
- Ofrece métodos similares a los de `Directory` pero con el beneficio de trabajar con un enfoque orientado a objetos.
- Esto permite realizar múltiples operaciones en un mismo directorio sin especificar la ruta en cada llamada.

6. Directorios

- Además, trabaja con `FileInfo` y `DirectoryInfo` para los archivos y carpetas contenidos (en lugar de `string`), lo que nos proporciona mayor control.
- El método `GetFiles` nos devuelve un array de `FileInfo` y el método `GetDirectories` nos devuelve un array de `DirectoryInfo`.
- Para crear una carpeta en la ruta especificada si esta no existe utilizamos el método `Create()`, y para crear subcarpetas `CreateSubdirectory(nombre)`.

6. Directorios

Ejemplo

```
1  DirectoryInfo tts55 = new DirectoryInfo("C:\\tts55");
2
3  foreach (FileInfo file in tts55.GetFiles())
4  {
5      Console.WriteLine(file.Name);
6  }
7
8  Console.WriteLine();
9
10 tts55.CreateSubdirectory("Nueva Carpeta");
11
12 DirectoryInfo[] directories = tts55.GetDirectories();
13
14 foreach (DirectoryInfo directory in directories)
15 {
16     Console.WriteLine(directory);
17
18     DirectoryInfo[] innerDirectories = directory.GetDir
19
20     foreach (DirectoryInfo innerDirectory in innerDirec
21     {
```

6. Directorios

- Al trabajar con `FileInfo` y `DirectoryInfo`, tenemos mayor control y operaciones disponibles, por ejemplo, podemos obtener el nombre de una subcarpeta sin la ruta completa o la extensión de cada uno de los archivos.



6. Directorios

Vídeo práctico

6. Directorios

Vídeo práctico - código

```
1  static void Main(string[] args)
2  {
3      Console.WriteLine(Directory.Exists("C:\\otra"));
4
5      Directory.CreateDirectory("nueva carpeta");
6      Directory.Delete("nueva carpeta");
7
8      foreach(var directory in Directory.GetDirectories("."),
9      {
10         Console.WriteLine(directory);
11     }
12
13     foreach(var file in Directory.GetFiles(".", "**.*", Se
14     {
15         Console.WriteLine(file);
16     }
17
18     DirectoryInfo directoryInfo = new DirectoryInfo(".");
19
20     foreach (var directory in directoryInfo.GetDirectories
21     {
22         Console.WriteLine(directory.FullName);
```

- El manejo de archivos es una tarea común en desarrollo, pero también es propensa a errores si no se siguen ciertas prácticas de programación segura y eficiente.
- Hemos visto que hay algunas excepciones muy típicas cuando trabajamos con archivos en C#.
- Vamos a ver algunas recomendaciones que mejorarán la calidad y robustez del código cuando se trabaja con archivos.

7. Buenas prácticas para el manejo de archivos

Validar archivos y carpetas

- Antes de leer, escribir o eliminar un archivo o directorio, verifica siempre su existencia:
- **Archivos:** Utiliza `File.Exists(ruta)` para comprobar si un archivo existe.
- **Directorios:** Usa `Directory.Exists(ruta)` para validar la existencia de una carpeta antes de realizar operaciones.
- Esta práctica evita excepciones como `FileNotFoundException` o `DirectoryNotFoundException`.

7. Buenas prácticas para el manejo de archivos

Manejo de Excepciones

- Como hemos visto, el acceso a archivos y directorios puede generar múltiples excepciones.
- Para manejar estos errores adecuadamente:
 - Usa bloques try-catch alrededor del código de manipulación de archivos.
 - Utiliza finally para liberar recursos o realizar cualquier limpieza, especialmente si has abierto flujos (Stream, StreamWriter, BinaryReader, etc).

7. Buenas prácticas para el manejo de archivos

Uso de `using` para Manejo Automático de Recursos

- Al trabajar con flujos (streams), usa bloques `using` para asegurar que los recursos se liberen automáticamente cuando ya no son necesarios.
- Esto es especialmente importante para evitar bloqueos de archivos.

7. Buenas prácticas para el manejo de archivos

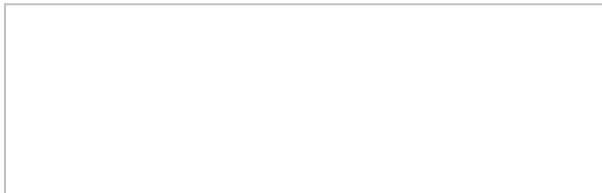
Usar rutas relativas

- Siempre que sea posible, evita rutas absolutas (por ejemplo, `C:\archivos\miarchivo.txt`) y usa rutas relativas.
- Esto hace que la aplicación sea más portátil y compatible con diferentes sistemas y entornos.

7. Buenas prácticas para el manejo de archivos

Diapositiva final

En este tema hemos aprendido a trabajar con archivos de texto, binarios y carpetas. También hemos aprendido como serializar y deserializar nuestros objetos, lo que nos permite persistir la información más allá de la ejecución del programa.



Speaker notes